Step 1: Create Simple Node.js App

1. Create a Project

```
C:\tirth-3093>mkdir load-balance-demo
C:\tirth-3093>cd load-balance-demo

C:\tirth-3093\load-balance-demo>npm init -y
(node:12140) ExperimentalWarning: CommonJS module C:\Program
Files\nodejs\node_modules\npm\node_modules\debug\src\node.js is loading ES Module
C:\Program Files\nodejs\node_modules\npm\node_modules\supports-color\index.js using
require().
Support for loading ES Module in require() is an experimental feature and might change at any
time
(Use `node --trace-warnings ...` to show where the warning was created)
Wrote to C:\Yash EX5\load-balance-demo\package.json:
{
  "name": "load-balance-demo",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

C:\tirth-3093\load-balance-demo>npm install express
(node:7004) ExperimentalWarning: CommonJS module C:\Program
Files\nodejs\node_modules\npm\node_modules\debug\src\node.js is loading ES Module
C:\Program Files\nodejs\node_modules\npm\node_modules\supports-color\index.js using
require().
Support for loading ES Module in require() is an experimental feature and might change at any
time
(Use `node --trace-warnings ...` to show where the warning was created)

added 65 packages, and audited 66 packages in 4s

22 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

2. Create server.js

Terminal 1:

C:\tirth-3093\load-balance-demo>notepad index.js

Edit index.js:

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;

app.get("/", (req, res) => {
  res.send(`Response from Server running on port ${PORT}`);
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

3. Run the application node server.js

```
C:\tirth-3093\load-balance-demo>node index.js
Server running on port 3000
C:\tirth-3093\load-balance-demo>node index.js
Server running on port 3000
```

4. Create a Folder Load balancing

Terminal 2:

C:\tirth-3093\load-balance-demo>notepad index1.js

Edit index1.js:

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3001;
app.get("/", (req, res) => {
  res.send(`Response from Server running on port ${PORT}`);
});
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 2: Run Multiple Servers

Open 2 terminals:

Terminal 1:

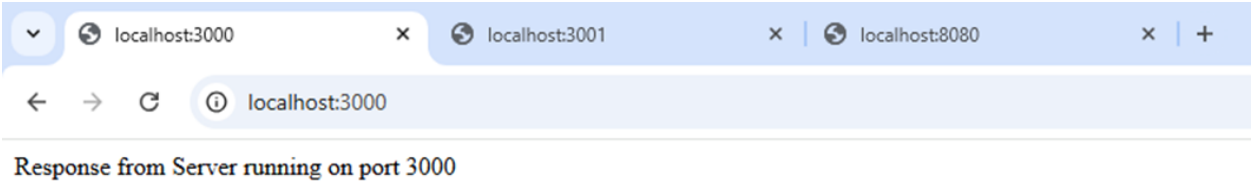
```
C:\tirth-3093\load-balance-demo>node index.js
Server running on port 3000
C:\tirth-3093\load-balance-demo>node index.js
Server running on port 3000
```

Terminal 2:

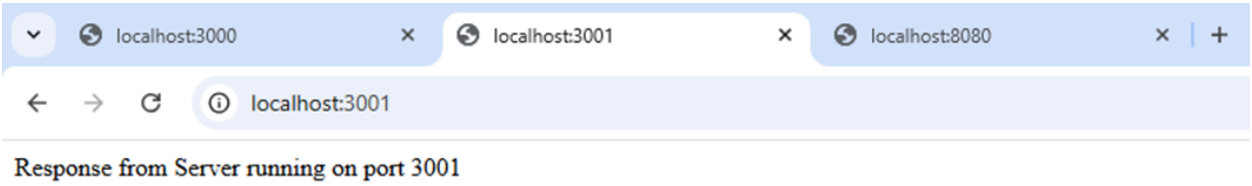
```
C:\tirth-3093\load-balance-demo>node index1.js
Server running on port 3001
C:\tirth-3093\load-balance-demo>node index1.js
Server running on port 3001
```

So we have now:

http://localhost:3000



http://localhost:3001



Step 3. Configure NGINX for Load Balancing

(Open nginx.conf and Modify http block)

```
events {
   worker_connections 1024;
}
http {
upstream myapp {
```

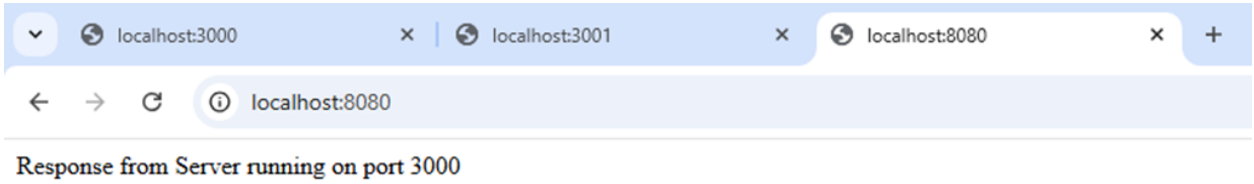
```
        server localhost:3000;
        server localhost:3001;
        }

    server {
        listen 8080;
        server_name localhost;

        location / {
            proxy_pass http://myapp;
        }
    }
}
```

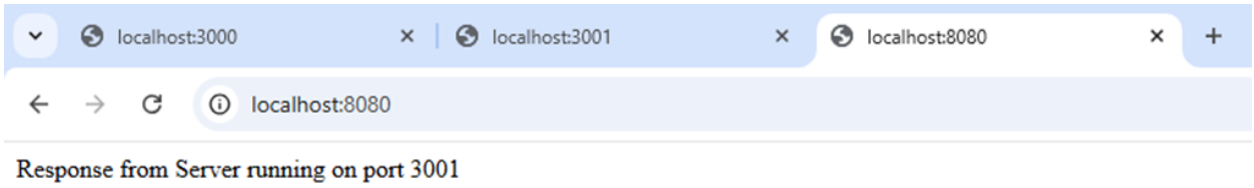
Step 4: Restart NGINX

Terminal 3:

```
C:\nginx-demo\nginx\nginx-1.28.2>nginx -s stop
C:\nginx-demo\nginx\nginx-1.28.2>nginx
```

Now open: http://localhost:8080
Refresh multiple times. You can check:





Step 5: Make Server 1 Slow

Modify Server running on PORT 3000:

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;
app.get("/", (req, res) => {
```

```
setTimeout(() => {
res.send("Slow Server - Port 3000");
}, 5000);
});
app.listen(PORT, () => {
 console.log(`Server running on port ${PORT}`);
});
```

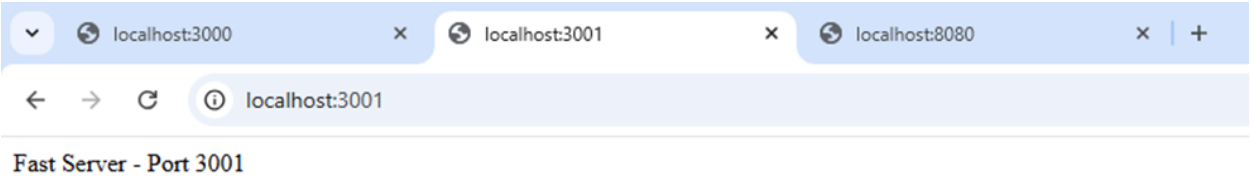
Keep Server 2 normal:

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3001;
app.get("/", (req, res) => {
res.send("Fast Server - Port 3001");
});
app.listen(PORT, () => {
 console.log(`Server running on port ${PORT}`);
});
```

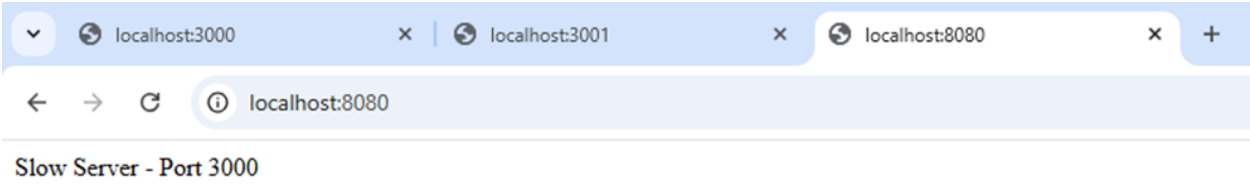
Step 6. Do Testing

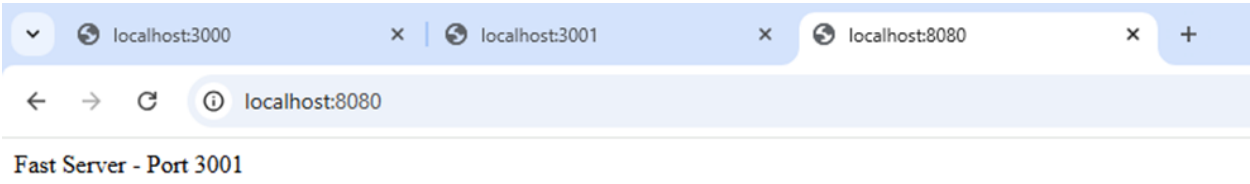
Server 1 need 5 second to load on 3000

Server 2 as it is Fast on 3000



Open: http://localhost:8080 Now refresh quickly multiple times.
Even though server 3000 is slow, NGINX still keeps sending every alternate request to it.
Server 1 need 5 second to load
Server 2 as it is Fast





Step 7: Now Change default algo of nginx(i.e. from round robin) to Least Connections and test again

Modify nginx.conf file:

```
events {
   worker_connections 1024;
}
http {
upstream myapp {
          least_conn;
          server localhost:3000;
          server localhost:3001;
          }
   server {
      listen 8080;
      server_name localhost;

      location / {
         proxy_pass http://myapp;
      }
   }
}
```

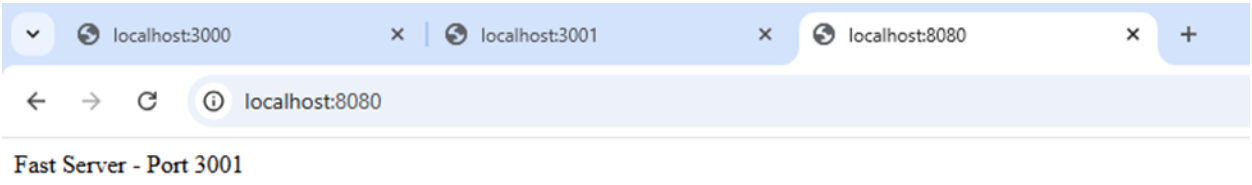
Restart NGINX:

```
C:\nginx-demo\nginx\nginx-1.28.2>nginx -s stop
C:\nginx-demo\nginx\nginx-1.28.2>nginx
```

Open: http://localhost:8080 Refresh quickly multiple times quickly.
When slow server (3000) is busy for 5 seconds:

It has an active connection.
So NGINX sees:
Server 3000 → 1 active connection
Server 3001 → 0 active connections

So it sends new requests to:
Fast Server (3001)



Fast Server - Port 3001